
Meristem: A Self-Invalidating Memory Substrate for Coding Agents

Kaustubh Sawant
parabkaustubh13@gmail.com

Abstract

1 Coding agents lose context at every session boundary: they re-derive why a con-
2 straint exists, re-litigate closed decisions, and burn tokens rebuilding orientation a
3 prior session already had. Retrieval-augmented approaches to this problem typi-
4 cally treat memory as a static index — a fact, once stored, is trusted until a human
5 or a timer says otherwise. We present Meristem, a memory substrate for coding
6 agents built around two properties we argue are necessary for code specifically
7 and largely unaddressed by existing agent-memory systems: *liveness*, in which a
8 stored fact carries a runnable predicate (a regex, an AST check, or a SQL query)
9 that re-verifies it against the current state of the repository rather than expiring it
10 on a timer or waiting for a new contradicting statement; and an *abstention gate*,
11 in which retrieval can return nothing rather than a confident but irrelevant top-*k*,
12 gated on a relevance signal we show empirically diverges from the graph-ranking
13 score used to order results. Meristem’s retrieval is an independent implementation
14 of HippoRAG’s Personalized-PageRank-over-a-knowledge-graph idea, applied to
15 a deterministically-ingested, typed, bi-temporal fact graph rather than an LLM-
16 extracted entity graph, and attached to an agent’s session through lifecycle hooks
17 that never call a language model in the hot path. On a synthetic multi-hop bench-
18 mark, Meristem answers all six questions across two structurally distinct corpora,
19 while grep, whole-file reading, a naive vector-RAG baseline, and labeled approxi-
20 mations of HippoRAG’s and Mem0’s approaches each solve at most two; an abla-
21 tion shows the graph traversal specifically, not just retrieval in general, is respon-
22 sible for Meristem’s advantage on facts that share no vocabulary with the query.
23 On Meristem’s own repository, it answers three real questions no grep or full-file
24 baseline can reach, at roughly 1/46 the token cost of reading whole files. We also
25 report telemetry from real, ongoing use across multiple independent production
26 repositories, showing the liveness mechanism has caught hundreds of real stale
27 facts in practice, not only in simulation. We report this evaluation’s real scope
28 alongside its results: the benchmarks are small and self-authored, the deployment
29 evidence comes from one practitioner’s projects rather than a multi-team study,
30 and two components discussed as design intentions — an explicit halt-before- hal-
31 lucination protocol and abstention under low confidence — are currently enforced
32 by convention and prompt discipline rather than by a code path that can refuse a
33 tool call outright.

1 Introduction

35 A coding agent’s session starts from nothing. It does not know why a particular check exists, whether
36 a design question was already settled, or that a plausible-looking fix was tried last week and rejected
37 for a reason that never made it back into any file. The agent’s only recourse is to re-read code: re-

38 derive the constraint, re-discover the decision, re-attempt the fix. On a small or young codebase this
39 is merely wasteful. On a mature one it is worse than wasteful — re-reading enough files to recon-
40 struct real context can, by itself, consume enough of a session’s context window that little of it is left
41 for the actual task, and nothing in a stateless session distinguishes “this was never considered” from
42 “this was tried and rejected,” so an agent can re-litigate a closed decision with the same confidence
43 it would bring to a genuinely open one.

44 Retrieval-augmented generation offers an obvious-seeming fix: store facts about the codebase, re-
45 trieve the relevant ones per query, inject them into context. In practice, most systems that take this
46 approach — surveyed in Section 2 — share two properties that we argue are specifically wrong
47 for source code, even where they are reasonable for the conversational memory these systems were
48 originally built for. First, a stored fact is trusted until something explicitly says otherwise: a timer
49 expires it, or a later conversational statement supersedes it, but nothing re-checks it against the one
50 artifact that can actually falsify a claim about code — the code itself. A fact about a database con-
51 straint that was true when it was written and has since been silently removed from the schema is, to
52 a memory system with no such check, indistinguishable from one that is still true. Second, retrieval
53 returns a ranked list whether or not anything on it is actually relevant; a ranking score answers “what
54 is best among these candidates,” not “is any candidate good enough to act on,” and the two questions
55 are easy to conflate when a system reports only the first.

56 This paper describes Meristem, a memory substrate built around addressing both properties directly
57 for the code-memory case, and reports what we can currently measure about whether doing so works.
58 Concretely, we contribute:

- 59 • **Liveness predicates**, a mechanism by which a stored fact carries a runnable check — a
60 regex, an AST query, or a restricted SQL statement — that re-verifies its claim against the
61 live repository, at query time and via a background sweep bounded to a fixed batch size
62 regardless of store size, so a fact whose code has since changed is dropped from results
63 rather than reported with unwarranted confidence (Section 3.4).
- 64 • **An abstention gate** built on a relevance signal we show, through direct measurement, di-
65 verges from the graph-ranking score used to order retrieval results — the two are uncor-
66 related enough that a nonsense query’s top-ranked result scored *higher* on average than a
67 genuine query’s, in one measurement against a real store (Section 3.5) — so that retrieval
68 can return nothing rather than a confident but irrelevant top- k .
- 69 • **An independent implementation** of HippoRAG’s Personalized-PageRank-over-a-
70 knowledge-graph retrieval idea (Section 2), applied to a deterministically ingested, typed,
71 bi-temporal atom graph rather than an LLM-extracted entity graph, integrated with the two
72 mechanisms above, and attached to an agent’s session lifecycle through hooks that never
73 call a language model in the hot path (Section 3.6).
- 74 • **An evaluation** against a purpose-built multi-hop synthetic benchmark, a real-repository
75 self-host benchmark, labeled approximate baselines for two named production systems,
76 three mechanism ablations, and telemetry from real, ongoing multi-repository deployment
77 (Section 4) — reported alongside an explicit accounting of what that evaluation does and
78 does not establish (Section 6).

79 We take the position, argued further in Section 5, that none of these four contributions is individually
80 unprecedented — typed knowledge graphs, PageRank-based retrieval, and relevance gating all have
81 prior art — but that the specific combination, applied with source code as the ground truth a memory
82 claim is checked against, is not one we found in the systems we surveyed. We report what is
83 measured and what is only argued from design as two different things throughout, including in the
84 abstract, because the alternative — a paper whose novel claims outrun its evidence — is a failure
85 mode this project’s own engineering practice was built specifically to avoid (Section 3.10), and we
86 hold the paper about it to the same standard.

87 2 Related Work

88 We place Meristem against four groups of prior work: the retrieval mechanism it builds on,
89 production-oriented agent memory systems, named systems working the same code-memory prob-
90 lem directly, and the general retrieval-augmented generation (RAG) literature it departs from.

Throughout, we distinguish an idea we adopt outright, an idea we approximate as a labeled evaluation baseline (Section 4), and an idea a named competitor implements that we do not.

Retrieval-augmented generation. RAG systems ground a language model’s output in retrieved external text rather than relying solely on parametric memory [Lewis et al., 2020]. The dominant retrieval mechanism in that literature is dense nearest-neighbor search over embedded chunks: relevance is a similarity score between a query vector and a document vector, computed independently per chunk. GraphRAG [Edge et al., 2024] departs from pure chunk retrieval by building an entity graph over a corpus and pre-summarizing communities of related entities, aimed at query-focused summarization over an entire corpus rather than pinpoint fact retrieval. Meristem shares GraphRAG’s premise that a graph structure carries information a flat chunk index cannot, but targets a different problem: retrieving a small number of precise, typed facts (a constraint, an invariant, an owner) relevant to a specific code location, not summarizing a corpus.

Personalized PageRank as a retrieval signal. Personalized PageRank (PPR) biases the classic PageRank random-walk algorithm [Page et al., 1999] toward a seed set of nodes rather than the whole graph [Haveliwala, 2002], so that a node’s score reflects its structural proximity to the query’s seeds rather than its global importance. HippoRAG [Gutiérrez et al., 2024] applies this directly to LLM memory: it extracts an entity-and-passage graph from a corpus using an LLM (open information extraction), seeds PPR with entities recognized in the query, and ranks passages by the resulting score. Meristem’s own retrieval (Section 3) is an independent reimplement of that same core idea — seed a query into a knowledge graph, rank by PPR rather than embedding similarity — applied to a different graph (typed code-memory atoms connected by 12 closed edge kinds, built by deterministic ingesters rather than LLM extraction) and extended with liveness re-validation and an abstention gate that HippoRAG does not have. No HippoRAG source was read or copied to build Meristem’s implementation; we cite it here as the origin of the algorithmic idea, not as a codebase Meristem derives from. Both projects are MIT licensed. Section 4’s `hipporag` baseline column is a lightweight, clearly labeled approximation of HippoRAG’s *approach* (regex-based pseudo-entity extraction and exact-string entity linking, in place of LLM open-IE) built for comparison inside our own harness — it is not the published HippoRAG system, and we do not claim parity with it anywhere results are reported.

Production agent-memory systems. Mem0 [Chhikara et al., 2025] and Zep [Rasmussen et al., 2025] are the two most visible production memory layers for conversational agents. Mem0 extracts atomic facts from a conversation with an LLM and, per new fact, uses a second LLM call to decide among ADD/UPDATE/DELETE/NOOP against existing memory — a judged, LLM-mediated write path. Zep’s Graphiti component builds a temporal knowledge graph over conversational and business data, tracking when a fact became and stopped being valid. Zep’s temporal model is the closest published analogue to Meristem’s own bi-temporal atom store (Section 3): both track a validity interval per fact rather than only a single timestamp. The difference is what triggers invalidation. Zep’s cascade invalidation (and, similarly, MinnsDB’s, below) fires when a *new stated fact* contradicts an old one — the mechanism a conversational agent needs when a user says something that supersedes what they said before. Meristem’s liveness predicates (Section 3) instead re-check a fact against the *current state of the actual source code* — regex, AST, or SQL-schema checks run against the live file tree, with no new statement required to trigger them. Both are conversational memory systems, not code-specific; neither re-validates a stored fact against a codebase. Section 4’s `mem0` baseline column is, similarly, a labeled stand-in for Mem0’s *approach* (heuristic fine-grained fact splitting and exact-normalized-text dedup, replacing its LLM-judged extraction and novelty check) — not the published system.

Named systems in the code-memory space. Three systems sit closest to Meristem’s own niche; two target coding agents specifically, and the field is crowded and moving quickly as of this writing [Ganguly, 2026]. TencentDB Agent Memory [TencentCloud, 2026] is, by adoption, the most credible named competitor: an MIT-licensed, Trendshift-featured project with an active community and multi-framework integration (Claude Code, Codex, CodeBuddy, and others), substantially more adopted today than Meristem. A source read of its public repository (no install; MIT, currently at v2.0.0) finds two concrete architectural differences from Meristem. Its retrieval combines BM25 and vector search via Reciprocal Rank Fusion — lexical and embedding signal, not graph-centrality ranking. And its results are count- or budget-capped rather than passed through a relevance floor

— there is no analogue to Meristem’s abstention gate (Section 3), which can suppress an entire retrieval when nothing clears a measured threshold, rather than always returning a ranked top- k . Its own repository’s only in-repo quantitative result is a PersonaMem benchmark (48% $\rightarrow$ 76% with the system enabled) — a general chat-memory benchmark, not a code-specific one; we report only that number, not the larger third-party press figures sometimes quoted for this project, which do not appear in the repository itself.

PROJECTMEM [Malo and Qiu, 2026] is, despite the similarly Tencent-adjacent-sounding name, an entirely unrelated, independent project from a different team (University of Utah) — worth stating plainly since the two are easy to conflate on first read. It is a local-first, event-sourced memory and judgment layer for coding agents: development is recorded as an append-only log of typed events (issues, attempts, fixes, decisions), projected into MCP-served summaries, with a deterministic pre-action gate that warns an agent before it repeats a previously failed fix. Architecturally, PROJECTMEM sits closer to Meristem than TencentDB Agent Memory does — both are local-first and per-repository rather than team- or cloud-shaped — but the memory model itself differs: an event-sourced log of what was tried, versus Meristem’s typed fact graph of what is currently true, re-checked against live code. PROJECTMEM’s own evaluation is a two-month, 207-event dogfooding study across ten real projects and a capability comparison against twelve contemporary memory systems; we do not attempt a head-to-head benchmark against it here.

MinnsDB [Minns-ai, 2026] combines a bi-temporal knowledge graph, a relational table engine, and a sandboxed WASM agent runtime in a single Rust binary, aimed at general agent workloads rather than code specifically. Its cascade invalidation is ontology-driven: edge behavior is defined in OWL/RDFS files loaded at startup, and asserting a new fact (their own documented example: a person moving city) automatically supersedes facts defined to depend on it — a gym-membership fact tied to the old city, in their illustration. Like Zep, this is invalidation triggered by a new assertion, not by re-checking source code; MinnsDB does not target software repositories or re-validate against a live file tree.

Positioning. None of the systems above combine the three properties Meristem’s design rests on (Section 3): liveness predicates that re-validate a stored fact against the current state of the actual code rather than expiring it on a timer or on a new contradicting statement; a bi-temporal atom store paired with typed, closed-vocabulary, weighted edges (as opposed to either dimension alone); and an abstention gate that can suppress a retrieval outright rather than always returning a ranked top- k . Graph-based retrieval over typed facts is, on its own, not a novel category — HippoRAG established the retrieval algorithm, and typed-fact graphs recur across the agent-memory space broadly. What we believe is distinctive is the specific combination, applied to source code as the ground truth a memory can be checked against. We state this as a design claim, not a demonstrated one: Section 4 reports what is actually measured, and Section 6 states plainly where evidence for this combination is still thin.

3 System Design

3.1 Overview

A coding agent’s session starts cold: it has no memory of why a constraint exists, which design decisions are already settled, or what was tried and rejected last time. It rebuilds context by re-reading files, which costs tokens and, on a mature codebase, can exhaust the context window before any task-relevant work begins. Worse, decisions already made get re-litigated, because nothing distinguishes “this was tried and rejected” from “this was never considered.”

Meristem addresses this with a knowledge graph of typed *atoms* — invariants, schema facts, decisions, conventions, dependencies, ownership, runtime facts, build recipes, and glossary entries — connected by typed *edges*. It is deliberately not a vector store: retrieval walks the graph with Personalized PageRank (PPR), seeded from embedding hits and the atoms located in the file currently open, so a structurally connected but lexically distant fact (a BLOCKS constraint, a MIRRORS partner two hops away) can outrank a closer but disconnected embedding match. The second half of the design is that facts expire: atoms carry *liveness predicates* that re-check a stored claim against the current state of the repository, so a fact whose code has since changed goes stale instead of continuing to be reported with unwarranted confidence. Both mechanisms are described below, followed by the

199 abstention gate that keeps retrieval honest about when it does not have an answer, the hook wiring
200 that attaches Meristem to an agent’s session lifecycle without a model call in the hot path, the write
201 path from code and conversation into memory, and the protocol for reconciling multiple developers’
202 stores without a shared server.

203 3.2 Atoms: a typed, bi-temporal fact store

204 An atom is a single typed claim about a codebase, identified by a content hash over its type, a
205 grouping key (`topic_key`), and a 50-word summary. The type taxonomy is closed at nine values,
206 enforced by a database constraint: *invariant* (a rule the code enforces), *schema_fact* (a table, column,
207 or constraint), *decision* (discussed below), *convention* (a project style rule, asserted only by hand),
208 *dependency*, *owner*, *runtime*, *build_recipe*, and *glossary*. *decision* carries a second, closed dis-
209 criminators, *decision_class* $\in \{\text{CONSTRAINT}, \text{CHANGE}\}$: every commit mints a *CHANGE* decision
210 as a historical record, while a *CONSTRAINT* forbids or shapes future work and is what retrieval and
211 the session-start digest surface by default — an early measurement on one real workspace found un-
212 differentiated commit-record decisions answering “what constrains this?” 76 times out of 76, which
213 is what motivated the split.

214 Atoms also carry a *tier* (*module*, *file*, or *symbol*) so that retrieval cost stays bounded as a repository
215 grows: a coarse pass runs first over module-level summaries, drilling into finer-grained children only
216 where a query’s PageRank mass concentrates, rather than ranking every symbol in a large repository
217 on every query.

218 Every atom is bi-temporal in the standard database sense [Snodgrass, 2000]: *valid_from*,
219 *valid_to* (NULL while live), *asserted_at*, and *superseded_by*. Reasserting a claim under an
220 existing topic key does not overwrite the row — it inserts the new atom and closes the prior one, pre-
221 serving the old claim as queryable history rather than destroying the fact that something was once
222 believed true. This is a deliberate design choice: it lets the system answer “didn’t we already decide
223 this?” and catch an agent re-litigating a closed constraint, and if the new claim genuinely diverges
224 from the old one (rather than being routine re-ingestion of, say, a new commit or schema snapshot),
225 the supersession also raises a *CONTRADICTS* edge between the two atoms so the conflict is surfaced
226 rather than silently overwritten.

227 Bi-temporal validity is not the staleness mechanism; that is the liveness predicate’s job (Section 3.4).
228 A live atom that fails its check is not closed or superseded — it stays live and keeps surfacing,
229 flagged as stale, because a liveness failure is evidence a recorded fact may no longer hold, not proof
230 that it has been formally retracted. Confidence, a real number in $[0, 1]$, feeds retrieval ranking with a
231 half-life decay measured from the atom’s most recent liveness pass, assertion, or validity start, so a
232 fact that keeps checking out stays fresh regardless of age while one with no runnable predicate sinks
233 as it goes unconfirmed.

234 3.3 The edge graph and PPR retrieval

235 Atoms are inert alone; a typed edge graph connects them. The schema closes edge kinds to twelve
236 values (seven are “fully wired” with a producing component described here; the remaining five
237 are declared in the schema and in retrieval’s weighting table but have no ingester emitting them
238 yet, an intentional latent-capacity gap rather than a hidden claim). Kinds: *MIRRORS* (the same
239 symbol name recurring across more than one registered repository root, symmetric), *REFERENCES*
240 (an atom’s summary text names a shared file path), *CO_CHANGED* (two files edited together in at
241 least three of the last 500 commits, weighted by co-commit count, symmetric), *OWNS* (person or
242 role to file, from git blame), *BLOCKS* (a decision that forbids changes under a path prefix, asserted
243 by a human or agent, never inferred), *ROLLS_UP* (a file or symbol atom to its module parent), and
244 *CONTRADICTS* (raised automatically when a supersession or a multi-developer merge detects two
245 live atoms disagreeing). Each edge carries its own producer-set weight (for example *BLOCKS* at
246 0.9, reflecting that a human asserted it deliberately) and a separate, retrieval-only kind weight used
247 during the PageRank transition, so a structurally important kind (*MIRRORS*, *BLOCKS*) propagates
248 more score mass per hop than a weaker structural signal (*OWNS*, *CO_CHANGED*).

249 Retrieval seeds a personalization vector from the top embedding matches for a query plus any atoms
250 structurally located in the files the agent currently has open, restricts the candidate universe to nodes
251 reachable within a bounded number of hops from those seeds (capping cost on a high-degree hub

the way tiering caps cost on a large repository), and runs power-iteration Personalized PageRank to convergence or an iteration cap. This follows the same shape as HippoRAG’s retrieval algorithm (Section 2): ranking is graph-flow through typed relationships rather than nearest-neighbor similarity alone, so a fact several hops away by a strong edge can outrank a closer but structurally isolated embedding match. Meristem’s implementation is independent — built over a differently-typed, deterministically-ingested graph stored in SQLite rather than an LLM-extracted entity graph held in memory — and extends the idea with the liveness and abstention mechanisms below, which HippoRAG does not have.

3.4 Liveness: self-invalidating against live code

A liveness predicate is a runnable check — *regex*, *ast*, or *sql* — that re-verifies an atom’s claim against the current repository. A *regex* predicate searches a named file for a pattern; an *ast* predicate re-parses a file and asks whether a named symbol is still declared, avoiding a word-boundary edge case a raw regex cannot handle for identifiers that begin with a non-word character; a *sql* predicate runs a restricted, single-statement SELECT against the atom store itself, for self-referential rules (for example, “every invariant has an owner”). A predicate is checked at two points: inline, when retrieval is about to return a candidate and its last successful check has aged past a freshness window, dropping a now-failing atom from the results entirely rather than returning a fact the system can no longer stand behind; and via a bounded background sweep that walks the N stalest live-predicate atoms per pass (a fixed, small N , independent of total store size), so sweep cost does not grow with the size of the repository. Because a passing check advances that atom’s own staleness clock, successive sweeps walk forward through the staleness frontier rather than repeatedly re-checking the same atoms. Facts with no runnable predicate — an asserted policy no grep or AST query could verify — instead carry a reconfirmation horizon; past it, the atom still surfaces but is marked unconfirmed rather than being silently trusted indefinitely.

A liveness target is recorded relative to a single base per atom, not always the workspace root: the workspace root itself for an atom whose registered root is the workspace root or a child subdirectory of it, and that atom’s own registered root when it sits outside the workspace root (a sibling, e.g. `roots = [".", "../service"]`); a traversal guard admits only a resolved path landing inside the workspace root or one of its registered roots. Getting this base wrong produces a false failure, not a false pass, and Section 3.10 reports two real instances of exactly that. The background sweep’s `watch -all` mode repeats its bounded batch until the stale frontier actually clears rather than running once, stopping the moment a batch makes no further progress — a genuinely failing predicate never advances its own clock, so an unconditional loop would otherwise never terminate.

3.5 The abstention gate

PPR score orders candidates within a single query but does not, by itself, indicate whether any of them actually answer it. A direct measurement against one workspace (198 replayed prompts over a 607-atom store) found top-1 PPR score for a nonsense query *exceeding* that of a genuine query on average, with no usable floor separating the two — PPR mass is a within-query ranking signal, not a cross-query relevance signal. Cosine similarity between the query embedding and each candidate’s own stored embedding separated the two conditions cleanly in the same measurement. Meristem’s retrieval therefore computes this relevance score for every candidate about to be returned and gates on it: candidates below a calibrated floor are suppressed, and a candidate with no relevance comparable under the querying embedding model (a cross-model mismatch) is treated as irrelevant by construction rather than passed through as unknown-but-safe. Separately, likely scaffolding — spent module-tier atoms and test-file symbol atoms that tend to embed well without answering the query — is filtered out. A query that clears no candidate returns nothing rather than a low-confidence top- k , and the suppression is logged with how close the nearest miss came, so an empty result is auditable as a deliberate abstention rather than indistinguishable from a broken retrieval path.

3.6 The ambient layer: session lifecycle hooks

Meristem attaches to an agent’s session through five lifecycle hook events rather than depending on the agent to remember to ask for context. *Session start* injects a digest: atom count, current branch, health findings, and the handful of atoms judged most load-bearing for the workspace. *Per*

304 *user turn*, a lightweight classifier routes the incoming prompt into one of a small number of intent
305 classes and injects a token-capped set of atoms from a fresh retrieval call — no model call in this
306 hot path; classification is regex-based and retrieval is the graph algorithm above. *After a file edit*, a
307 watcher re-runs the liveness predicates of any atom whose target names the touched file and surfaces
308 a warning if one now fails, advisory only since the edit has already landed. *At the end of a turn*,
309 a pattern-based extractor scans the conversation for the grammar of a standing rule — universal
310 quantifiers, exceptions, prohibitions, policy modals — and queues candidates for human review
311 rather than writing anything directly into memory. *Approaching a context compaction*, a handoff is
312 written: the current task, atoms consulted and drafted, open questions, and what was already tried
313 and did not work, so the next session resumes from that record instead of from nothing. Every hook
314 invocation writes a heartbeat record before deciding whether to act at all, specifically so that “wired
315 but doing nothing” is distinguishable from “never wired,” a distinction whose absence let this same
316 ambient layer sit silently dead on its own authors’ machine for eight days after an unrelated rename,
317 caught only by an unrelated health-check complaint rather than by any test.

318 Because sessions run for days, a session-start notice goes stale mid-session. The housekeeping
319 notice — index drift behind HEAD, and captured facts awaiting human review — therefore re-surfaces
320 on a later turn, once per change in its content and once per stale episode rather than on every prompt,
321 from per-session state kept in the per-user state directory rather than in the repository. Session
322 start also treats freshness as a read-path concern: when the index is behind, it spawns a detached,
323 debounced, lock-guarded background sync and keeps the staleness notice, annotated that a sync has
324 started, rather than hiding it because a repair is underway. The hook never waits on the sync.

325 A session running inside a linked `git worktree` add checkout, or in a subdirectory of a workspace,
326 has no `.meristem/` directly under its own directory to find, and previously fell back silently to “not
327 a workspace” — no recall, no capture, nothing surfaced to say so. Resolution now also tries the `git`
328 toplevel and the main checkout backing a linked worktree, each behind a short, bounded timeout,
329 since an earlier unbounded `git` probe once stalled a hook invocation for 60 seconds when `git` itself
330 hung.

331 3.7 Access beyond the ambient layer: the MCP server

332 The five hooks above are specific to one agent product’s session-lifecycle events. Every mechanism
333 they call on — retrieval, liveness re-validation, graph traversal, and the write path (Section 3.8) —
334 is also exposed through a standard Model Context Protocol (MCP) server, usable from any MCP-
335 capable client over the same three transports MCP clients already expect (a local subprocess, or
336 HTTP/SSE for a remote or web agent). The server exposes eight tools: a retrieval call mirroring
337 the query mechanism of Section 3.3 (returning suppressed and scaffolding-hidden counts alongside
338 any surfaced atoms, so an empty result reads as a deliberate abstention rather than an unexplained
339 failure); a direct-write and a review-queued write mirroring the two manual routes of Section 3.8;
340 supersession; graph-neighbor traversal; an atom’s full provenance trail (source, a liveness re-check
341 run at request time, never cached, and its edges); and a health check. None of these calls the language
342 model client any differently than the CLI equivalents they mirror. What does not travel over MCP
343 is the ambient layer itself — digest injection, per-turn routing, the invariant watcher, and automatic
344 handoff stay tied to Claude Code’s specific hook events (a client without them calls the retrieval
345 and write tools explicitly instead of receiving auto-injected context), which is the generalization gap
346 Section 5 returns to.

347 3.8 The write path

348 Three routes populate the atom store, and all three converge on one write function, so that “does
349 this claim already exist under this topic” is answered identically regardless of how the claim arrived.
350 *Automatic ingestion* runs a fixed set of deterministic adapters over each configured repository root:
351 README/manifest/schema/invariant extraction from declared constraints, one decision atom per
352 commit, symbol and module extraction (tree-sitter where available, falling back to a per-language
353 regex), and ownership from `git blame`. None of these adapters call a language model; a constraint
354 atom, for instance, is only ever minted from a *declared* database constraint the extractor can point a
355 liveness predicate at, not an inferred business rule. *Manual* assertion is a human or agent explicitly
356 recording a decision, optionally with a liveness predicate and with BLOCKS edges to every already-
357 ingested atom under a given path prefix, making the constraint reachable by graph traversal from

the code it governs. *Captured* facts come from the end-of-turn conversation extractor described above: a candidate is deliberately not retrievable until a human accepts it through a review queue, an asymmetry stated directly in the system’s own design rationale — a wrong candidate costs one keystroke to reject, while a wrong fact written directly into the graph is retrieved and believed later. Extraction is intentionally pattern-based rather than a further model call, trading extraction precision for keeping the write path as cheap and fast as the read path. Two refinements keep that review queue usable and safe. A deterministic, local trust guard scans for secrets and personal data: a unit that trips it is never proposed for capture, and an atom that carries one is withheld from the shared, git-mergeable export (Section 3.9); findings are reported by atom id and kind, never by value. We added it because, in practice, free-text conversation and source comments do carry such material. Separately, an agent may *suggest* a disposition for a pending candidate (accept or reject, with a reason), shown beside the item, but only a human disposes: extraction proposes, a human disposes (Section 4.4).

3.9 Multi-repository synchronization

Because two developers’ concurrent writes to a shared binary database file are a git conflict resolvable only by discarding one side, the shared substrate leaves each machine as sorted, line-oriented JSON rather than as a database file: an export of exactly the tables meaningful across machines (atoms, their summaries, edges, and edge evidence), with per-machine operational tables such as job queues and retrieval logs excluded by design. Merging two exports is a commutative, associative, per-field policy chosen so the result never depends on which side is treated as “ours” — for example, a validity end-time prefers whichever side already closed the row, since a row cannot be un-closed, and a liveness kind prefers whichever side recorded the more precise predicate type. A genuine disagreement, as opposed to mechanical divergence on the same row — two developers asserting different claims under the same topic key while offline — is not treated as a field-merge problem: import closes every topic down to one live survivor and raises a CONTRADICTS edge to each closed duplicate, surfacing the disagreement rather than letting import order decide it silently.

Keeping the index level with HEAD depends on git hooks firing, and two defects in the first design made them silently inert. A managed hook named a single workspace, so the last workspace to install against a shared repository won, and a renamed or removed workspace left a stale path that made the hook’s existence guard exit on every commit; nothing reported it. Separately, `post-commit` does not fire on merge, pull, or rebase, which is exactly how a main checkout’s HEAD advances in a branch-then-merge workflow, so the index drifted until a periodic job caught up, visible only as a session-start drift warning. Managed hooks now carry a list of workspaces (a legacy single-workspace hook is upgraded in place and dead entries are pruned at install), synchronization also runs on `post-merge` and `post-rewrite`, and a doctor check (`hooks.git`) warns about missing or misattributed hooks. Synchronization never alters captured facts awaiting review.

3.10 Design principles

Three principles recur across the mechanisms above, and are stated explicitly in the system’s own source rather than reconstructed after the fact for this paper. *The language model stays outside the core loop*: ingestion, retrieval, liveness checking, and conversation extraction all run without a model call, because a hook that must wait on a model response before deciding whether to act at all is a hook that can stall a turn — and, empirically, a hook that silently does nothing under load is indistinguishable from one that was never wired in the first place. *Failures are surfaced loudly, not silently*: three real incidents — an edge-density check that warned instead of failing while two production workspaces ran with an empty graph for months, a hook wired into the agent’s configuration but pointed at a since-renamed binary producing no evidence anywhere for eight days, and a computed-but-never-rendered review-queue count — share the same shape: a check already existed, and its severity or visibility, not its logic, let the failure go unnoticed. The fix in each case was not smarter detection but making an existing signal impossible to miss. A related failure mode is not a check going silent but firing confidently and wrong: two path-resolution bugs (Section 3.4) made liveness predicates report failed against the wrong file entirely, 224/224 in one real workspace and 423/423 in another — a stale verdict as loud as a correct one. Neither was caught by a unit test; both surfaced only because the health check’s own aggregate stale count looked implausibly high on an actively maintained workspace. Both were verified fixed against the affected store (423 failed → 423 verified, its 24 genuinely stale atoms still failing) and across 13 real workspaces before/after,

413 with no other verdict changing. *A human disposes; the system never auto-accepts a captured fact:*
414 the review queue exists specifically so that low-precision, cheap extraction can be attempted at all,
415 because the cost of a false positive (one keystroke) and the cost of a wrongly-written fact (retrieved
416 and believed indefinitely) are not symmetric, and only a write path with a human gate in front of it
417 can afford to be permissive on the extraction side.

418 4 Evaluation

419 4.1 Experimental setup

420 We evaluate along three axes: a synthetic multi-hop benchmark designed so that a purely lexical
421 baseline structurally cannot win, a real-world benchmark against Meristem’s own self-hosted mem-
422 ory of its own source repository, and telemetry from real, ongoing use across multiple independent
423 production repositories. All synthetic-corpus and self-host numbers below are produced by a single
424 benchmark harness (`tools/eval_harness.py` in the project repository) that builds its own dis-
425 posable git repository, runs the real ingestion, edge-discovery, embedding, and retrieval pipeline in-
426 process, and measures every method against the same questions and the same file set — no method
427 is evaluated against a more favorable view of the corpus than any other.

428 **Synthetic corpora.** Two six-question corpora probe structurally different retrieval paths. *Schema*
429 questions name a symbol in one file; the governing constraint lives in a separate file with no shared
430 vocabulary, reachable only through a `CO_CHANGED` edge (files repeatedly committed together) or
431 a direct embedding hit. *Policy* questions ask about a decision asserted directly with `meristem`
432 `decide --blocks`, mirroring how a human actually records a constraint — never derived from
433 any file’s text — reachable only through a `BLOCKS` edge. Policy questions are constructed so their
434 wording shares no word stem with the governing decision’s own text (enforced by a dedicated test
435 in the project’s suite), so `grep` and `full-file-read` have no literal string to find and a bag-of-words or
436 embedding-only method has no lexical overlap to exploit either; only graph traversal reaches the
437 answer.

438 **Methods compared.** `meristem` is the full system (Section 3). `grep` is a literal substring search
439 with context lines. `full_file_read` reads whole the files `grep` matched. `vector_rag` chunks
440 every file into overlapping windows and cosine-ranks them with the same embedder the corpus uses
441 — none of Meristem’s graph, PPR, liveness, or gating machinery. `hipporag` and `mem0` are clearly-
442 labeled, lightweight approximations of HippoRAG’s and Mem0’s *approach* (Section 2): regex-based
443 pseudo-entity extraction and exact-string linking plus an independently-implemented PPR power it-
444 eration for `hipporag`; heuristic fine-grained fact splitting and exact-normalized-text dedup plus per-
445 fact embedding for `mem0` — neither makes a language-model call, since real fidelity to either pub-
446 lished system would require one (open information extraction for HippoRAG, LLM-judged fact nov-
447 elty for Mem0), which would introduce API cost, network calls, and non-determinism this harness
448 is built to avoid. Neither is compared as if it were the published system, and we do not report either
449 as evidence about HippoRAG’s or Mem0’s own real-world performance — only about what a graph-
450 shaped and a flat-fact-shaped alternative do on this benchmark. Three ablations flip a single mecha-
451 nism inside Meristem’s own pipeline relative to its default configuration: `embed_only` disables PPR
452 and ranks purely by embedding cosine; `liveness_on` enables inline liveness re-verification (off by
453 default in this harness, since the synthetic corpus is static); and `gate_off` disables the abstention
454 gate’s scaffolding filter (Section 3.5).

455 **Reproducibility.** Every synthetic-corpus commit timestamp is pinned rather than taken from wall-
456 clock time, and every ranking computation uses an explicit tie-break and a fixed iteration order, so
457 that two runs of the full comparison — including a run under a different Python hash seed — produce
458 identical recall, token, and ranking numbers. We verified this directly for this paper (two repeated
459 runs plus one run under an altered hash seed, diffed after normalizing an incidental, intentionally-
460 random temporary-directory name that some methods’ trace strings embed but which is not part of
461 any reported number).

Method	Overall recall	Policy recall	Schema recall
<code>meristem</code>	6/6	3/3	3/3
<code>grep</code>	0/6	0/3	0/3
<code>full_file_read</code>	0/6	0/3	0/3
<code>vector_rag</code>	1/6	0/3	1/3
<code>hipporag</code> (HippoRAG-inspired)	0/6	0/3	0/3
<code>mem0</code> (mem0-style)	2/6	0/3	2/3
<code>embed_only</code> (ablation)	1/6	0/3	1/3
<code>liveness_on</code> (ablation)	6/6	3/3	3/3
<code>gate_off</code> (ablation)	5/6	3/3	2/3

Table 1: Synthetic corpus recall by method. `meristem` is the only method to solve every policy question, by construction the ones no lexical or embedding-only method can reach.

4.2 Synthetic corpus results

`meristem` solves all six questions; every baseline that does not traverse the graph fails every policy question outright, as designed — there is no file, chunk, or entity co-occurrence containing the answer for those three. `vector_rag` and `mem0` pick up one and two schema questions respectively through embedding similarity where the query happens to share vocabulary with a nearby fact; `hipporag`’s approximation solves none, since this corpus’s schema and policy links exist only as CO_CHANGED/BLOCKS graph signal, invisible to a lexical entity-co-occurrence graph built without embeddings.

The ablations isolate what each mechanism contributes. `embed_only` — Meristem’s own retrieval with PPR replaced by raw embedding cosine-ranking — drops from 6/6 to 1/6, losing all three policy questions outright: with the graph traversal removed, the policy corpus becomes exactly as unreachable to Meristem’s own embedder as it is to `vector_rag`, which is the ablation’s intended measurement, not a regression. `liveness_on` matches the default configuration exactly, which is an honest non-result rather than a null one: nothing in this static synthetic corpus is actually stale, so inline re-verification has nothing to catch; what it does change, checked directly against the retrieval internals rather than the recall number, is that every candidate’s liveness state is confirmed `verified` instead of left unchecked. `gate_off` drops one schema question by removing the scaffolding filter, which shifts probability mass toward module-tier distractors that would otherwise be dropped before ranking — disabling a precision mechanism costs a small amount of recall here, the trade the gate exists to make in the other direction.

4.3 Real-world evaluation: Meristem’s own repository

Method	Recall	Avg. tokens
<code>meristem</code>	3/3	182
<code>grep</code>	0/3	90
<code>full_file_read</code>	0/3	8,342

Table 2: Real-repository evaluation against Meristem’s own live, self-hosted memory store (2,039 atoms at time of evaluation), three hand-verified questions about the project’s own real commit and schema history. All three rows are a single 2026-09-28 run (see text and `paper/data/README.md`).

Unlike the synthetic corpora, these three questions are not constructed to defeat a naive baseline — they are ordinary questions about a real repository’s real content, verified individually against atoms actually captured from real development activity. `grep` and `full_file_read` nonetheless fail all three: one question is phrased as a natural-language “why,” with no literal string anywhere in the repository to match; the other two ask about a constraint (a NOT NULL invariant, a capacity cap) that exists in the schema but is never spelled out as a code comment next to the symbol it governs, so `grep` on the symbol’s own file finds nothing and reading the whole file does not help. `full_file_read` costs roughly 46× `meristem`’s token budget on this corpus and is still wrong on every question. Re-run for this paper, `meristem`’s recall stays 3/3 while its average token cost moves within a narrow

band rather than tracking store size monotonically: 182 tokens nine days before the 2026-09-12 run’s 191, and 182 again at 2,039 atoms 16 days later (Table 2). `grep` and `full_file_read` are from a clean 2026-09-28 re-run: an earlier run let both baselines match `paper/`’s own committed harness output and question prose instead of real source, the same self-match shape `NOISE_DIRS` already excluded `.claude` worktrees and a prior `eval_results` directory for; `paper` now joins that list. With the source fixed, `grep` costs the same 90 tokens as 2026-09-12 (its two matched files are unchanged where `grep`’s context window looks), while `full_file_read` moves slightly, 8,248 to 8,342 tokens, as those files grew a little over the same 16 days.

We report only three questions here, and we say so plainly rather than letting the clean recall numbers imply more than they support: this is a demonstration on one repository, not a benchmark with statistical power, and it is the one real-world corpus in this paper that can be reported under the project’s own name, since Meristem’s repository is the paper’s subject rather than third-party data (Section 6 returns to this constraint).

4.4 Real-world deployment telemetry

Beyond the self-host benchmark above, Meristem has been running its own ambient hooks (Section 3.6) continuously across multiple independent real software projects, not only its own repository. Because those projects are private and cannot be named or described beyond aggregate counts (Section 6), we report only totals with no per-project breakdown, computed by a read-only script over each workspace’s own store (`paper/data/fleet_telemetry.py` in the project repository). Across thirteen independently initialized workspaces on one practitioner’s real projects, ten show retrieval activity and six have both at least two weeks of history and recent use, ranging from just over two weeks to nearly six weeks of continuous operation. Retrieval logging shows 2,153 real query events across those ten stores. The manual and reviewed write paths (Section 3.8) have together produced 53 accepted constraint-class decisions across every workspace combined, up from 39 measured 16 days earlier in the same deployment. On liveness specifically: Meristem’s own self-hosted workspace alone recorded 420 real liveness-predicate catches (418 regex, 2 AST) over its usage window — a predicate that had previously passed, failing on re-check against genuinely changed code, exactly the mechanism Section 3.4 describes, observed in practice rather than only argued from design; a separate workspace’s liveness sweep independently caught six more genuinely stale decision atoms in a single run. Both of these two workspaces use a single root (`roots = ["."]`), so neither catch count is affected by the multi-root path-resolution bugs Section 3.10 describes. Per-turn injected token cost, logged automatically on every retrieval call, now averages 63.9 tokens against a 3,000-token per-turn cap, over 2,153 logged real-session retrievals — no longer the thin, 14-sample estimate reported previously. That blended figure spans two behaviors: the gate injects nothing on about 38% of these turns (819 of 2,153), while the rest — turns that inject anything — average 103.1 tokens.

Capture precision from human review labels. Every accept or reject in the review queue is a label on the extractor. Across 121 labelled candidates from real workspaces (26 accepted, 95 rejected), extraction precision was 21.5%, and all 121 still passed the extractor. Deterministic rules, together with learning from rejections by dropping candidates whose token Jaccard similarity to an already-rejected candidate is at least 0.6, raise precision to 27.8%: 30 of the 95 rejected candidates are removed and 1 of the 26 accepted is lost. That single loss is why the addressed-to-agent rule is a score penalty rather than a drop. The remaining rejections are product requests that are lexically indistinguishable from accepted requirements, so lexical rules cannot separate them (Section 6). These counts come from private stores and are replayed locally, not reproducible from the public repository.

We are explicit that this telemetry demonstrates the mechanisms fire on real, unmodified developer activity — it is not a controlled comparison against a baseline, there is no held-out ground truth for what these workspaces’ correct decision count or catch rate “should” be, and it comes from a single practitioner’s projects rather than a multi-team or multi-organization deployment.

4.5 Scope and threats to validity

Three limitations apply specifically to the evaluation methodology above, independent of the broader limitations of the system itself (Section 6). First, both synthetic corpora are small (six questions

each) and were authored by the same team that built the system being evaluated, which is a normal way to build a controlled benchmark but not a substitute for an external, adversarially-constructed one. Second, the hipporag and mem0 baselines are approximations of those systems’ *approach*, not the published systems themselves, and every number reported for them should be read as a statement about a graph-shaped or flat-fact-shaped alternative built inside this harness, not as a reproduction of HippoRAG’s or Mem0’s own published results. Third, the harness supports an embedder ablation (hash embedding vs. the real bge-small-en-v1.5 model vs. a reciprocal-rank-fusion hybrid of the two), which we do not report here: a real workspace’s store is embedded with exactly one model at a time (Section 3.5’s cross-model handling assumes this), so there is only ever one ranking to compare against real data, and no committed run of this ablation exists for either corpus in this paper’s evidence directory. We name the gap rather than report a number we have not actually produced.

5 Discussion

What the evaluation actually shows. The synthetic benchmark (Section 4.2) demonstrates a narrow but specific claim well: when an answer is reachable only through a typed graph edge and shares no vocabulary with the query, graph traversal reaches it and every method we compared against — including our own embedding-only ablation of Meristem’s own retrieval — does not. That is a real, mechanistic result, not a proxy for one. It is also a deliberately favorable setting for the claim being tested: real codebases are not constructed so that every important fact is vocabulary-disjoint from every question about it, and the real-repository result (Section 4.3) is closer to that harder reality, though still only three questions. The deployment telemetry (Section 4.4) is a different kind of evidence again — not a controlled comparison, but a report that the liveness mechanism specifically has fired hundreds of times on real, unprompted code changes across real projects, which is the strongest evidence in this paper that the mechanism does something in practice, not only in a benchmark built to exercise it.

Measured versus argued from design. We have tried to keep these visibly separate rather than letting the paper’s structure blur them. The liveness mechanism, the abstention gate’s motivating measurement, and the ablation results are measured. The scaling argument in Section 6 — that liveness sweeps and PPR retrieval are architecturally bounded, independent of total store size — is a claim about the code’s structure, verified by reading it, not a claim backed by a benchmark at the scale where it would matter; we say so explicitly rather than letting a real architectural property read as an empirical one. The same discipline applies to the paper’s central positioning claim (Section 2): that the specific combination of liveness, bi-temporal validity, typed weighted edges, and an abstention gate is not one we found elsewhere. That is a survey claim about the systems we read, not a formal impossibility result, and it will be wrong the moment a system we did not survey turns out to combine the same properties — a real risk given how quickly the field surveyed in Section 2 is moving.

The cost of staying deterministic. Meristem’s write path (Section 3.8) deliberately avoids a language-model call for ingestion, liveness checking, or conversation extraction, trading extraction precision for latency and cost predictability. The design rationale — a hook that must wait on a model call before deciding whether to act at all threatens the one property the ambient layer exists to provide — is sound as far as it goes, but it is a real tradeoff, not a free one: a marker-based extractor over conversation text will miss a standing rule phrased in a way its patterns do not cover, where an LLM extractor plausibly would not. The review-queue gate (Section 3.8) is what makes this tradeoff acceptable rather than merely cheap — a missed extraction costs nothing beyond the fact not being captured, whereas a wrongly *accepted* one would cost a false memory retrieved as truth later — but it does mean Meristem’s captured-fact recall is bounded by pattern coverage in a way a model-mediated system’s would not be. We view this as the correct choice for this project’s stated design principles, not as a costless one.

Beyond Claude Code. Every mechanism described in Section 3 except the ambient hook layer itself is exposed through a standard MCP server (Section 3.7) and usable from any MCP-capable client, so the retrieval, liveness, and write mechanisms this paper evaluates are not architecturally tied to one agent product. The session-lifecycle automation — digest injection, per-turn routing,

the invariant watcher, automatic handoff — currently is tied to Claude Code’s specific hook events, and we have not evaluated what porting that layer to another agent’s lifecycle events would cost or change; we flag this as future work rather than as an already-solved generalization.

6 Limitations

We list what we know is incomplete or unproven, in the same spirit as the engineering practice this system is built around (Section 3.10): a limitation stated plainly is more useful to a reader than one left for them to discover.

Evaluation scale. The synthetic benchmark is six questions across two corpora, authored by the same people who built the system being evaluated; the real-repository benchmark is three questions against one repository. Both are useful for testing a specific mechanistic claim under controlled conditions, and neither has the statistical power or external authorship that a benchmark intended to establish general retrieval quality would need. We are not aware of a larger, independently constructed benchmark for this exact problem (typed, code-grounded agent memory) to compare against; building one is future work, not something this paper substitutes for with a larger synthetic corpus of the same kind.

Deployment scope. The real-world telemetry in Section 4.4 comes from one practitioner’s real projects, not a multi-team or multi-organization deployment. The multi-repository synchronization protocol (Section 3.9) is designed for concurrent, offline-capable multi-developer use and is exercised end to end by the project’s own test suite and by one real two-repository onboarding, but we have not run it under real concurrent multi-developer load, and we make no claim about what happens to conflict-resolution behavior or review-queue volume at team scale. Multi-root support within a single workspace (Section 3.4) is on firmer ground: real workspaces now exercise both a child-subdirectory-shaped and a sibling-shaped multi-root configuration, not only the test suite’s own fixtures, which is how the two liveness path-resolution bugs (Section 3.10) were found.

Approximate, not head-to-head, competitor comparisons. The *hipporag* and *mem0* baselines in Section 4.2 are labeled approximations of those systems’ retrieval *approach*, not the published systems run against our benchmark, and the architectural comparisons against TencentDB Agent Memory, PROJECTMEM, and MinnsDB in Section 2 are based on reading each project’s public source and documentation, not on running any of them against our benchmark or having them run Meristem’s. Every claim in this paper about how those systems differ from Meristem should be read as a survey claim, current as of when we read each project, not as an empirical result.

Capture precision is still low. Even with the rules and near-duplicate learning of Section 4.4, most proposed candidates are rejected; what remains are product requests that read exactly like accepted requirements. Closing that gap needs learned, per-workspace signals, which we leave as future work.

Scale is argued, not measured. PPR retrieval is hop-capped and liveness sweeps are batch-bounded specifically so that per-query and per-sweep cost do not grow with total store size (Sections 3.3, 3.4) — an architectural property, verifiable by reading the implementation. We have one real data point in that direction (a full health-check pass completing in under four seconds against roughly eleven hundred atoms in one real store) and no stress test at the scale — tens of thousands of atoms, repositories with hundreds of thousands of lines — where a bounding argument like this would actually be tested against reality rather than against its own design. The symbols ingester has a per-run cap, so coverage of a large repository is incremental and has not been benchmarked at full coverage. Windows support is unverified.

Halt-before-hallucination is a protocol, not an enforced mechanism. Meristem’s handoff and session-digest machinery (Section 3.6) is designed around an explicit contract that an agent should halt and ask rather than proceed past a stated confidence threshold. As currently implemented, that contract is a string injected into the agent’s context, not a code path capable of refusing a tool call. Nothing in this system currently blocks an action the way, for instance, a liveness check can drop a stale fact from a result set before it is ever returned. We state this because the distinction matters

for how the claim should be read: as a protocol an agent is asked to follow, which is falsifiable and useful, not as an enforcement guarantee, which it is not yet.

Coverage gaps by design, not yet closed. Five of the twelve schema-defined edge kinds (IMPLEMENTS, DERIVED_FROM, DEPENDS_ON, LOCATED_IN, and SUPERSEDES as a first-class edge rather than an atom field) have no producing component yet — declared capacity in the schema and in retrieval’s weighting table, not a hidden claim, but also not evidence the corresponding relationships are currently discoverable by graph traversal. The abstention gate’s relevance floor (Section 3.5) was calibrated from one measurement against one real store under one embedding model; we have not re-derived or validated it against a second independent store or a second embedding model, so we cannot yet say how sensitive the floor is to the workspace it was tuned on.

Two more health-check false positives. Liveness freshness (Section 3.4) is purely time-based: an unchanged repository on a machine asleep for more than a day reads as 100% stale and the session digest reports the store as degraded, even though nothing it checks has actually changed. Automatic synchronization is best-effort: a machine with no managed hook still drifts until the next trigger, which the doctor check reports rather than prevents. Separately, a root removed from a workspace’s configuration stays registered in the store’s repository-state table and is reported as “never ingested — the store is empty” even when the store holds hundreds of live atoms from the remaining root. Both are false positives of the same class Section 3.10 already discusses — a verdict that fires loudly and is simply wrong — and neither is fixed as of this writing.

Privacy constrains what can be shown. The real-world deployment evidence in Section 4.4 is aggregated across private, unnamed repositories by necessity, not by choice: we cannot publish a per-repository breakdown, example captured facts, or qualitative detail about what the liveness mechanism actually caught, because doing so would risk identifying the underlying private projects. This makes that part of the evaluation harder for an outside reader to independently audit than the synthetic and self-host results, once the evaluation harness is published in the public repository. This constraint is not hypothetical: while assembling a redistributable package, a grep gate for private terms — run as a pre-packaging check to catch exactly this before anything left the lab — found a private project’s name and real people’s names inside illustrative source comments and docstrings of the capture module, left over from early development before that module’s example text was replaced with synthetic examples of the same grammatical shape. An earlier, partial pass had already replaced one such example but missed the rest; all were found and replaced before the redistributable package was assembled.

7 Conclusion

We presented Meristem, a memory substrate for coding agents built around two properties we argue are necessary for code memory specifically: liveness predicates that re-verify a stored fact against the current state of the repository rather than trusting it until a timer or a new statement says otherwise, and an abstention gate that can return nothing rather than a confidently ranked but irrelevant answer. Its retrieval mechanism is an independent implementation of HippoRAG’s Personalized-PageRank-over-a-knowledge-graph idea, applied to a deterministically ingested, typed, bi-temporal atom graph and attached to an agent’s session without a language model in the retrieval or liveness hot path. On a synthetic multi-hop benchmark and on a real-repository self-host benchmark, Meristem reaches facts that grep, whole-file reading, a naive vector-RAG baseline, and labeled approximations of two named production memory systems do not, at a fraction of the token cost of reading whole files; an ablation isolates that this advantage specifically comes from graph traversal, not from retrieval in general. Telemetry from real, ongoing use across multiple independent production repositories shows the liveness mechanism catching real stale facts in practice, not only in a benchmark built to exercise it.

We have tried throughout to report what is measured and what is only argued from a reading of the system’s own design as two different things, including where that means stating plainly that a mechanism — most notably, halt-before-hallucination — currently exists as a protocol an agent is asked to follow rather than as code that can refuse an action. The evaluation in this paper is small: two purpose-built synthetic corpora, three real questions against one repository, and deployment telemetry from one practitioner’s projects rather than a multi-team study. Closing that gap — a larger,

independently constructed benchmark; a multi-team or multi-organization deployment; a head-to-head run against the published HippoRAG and Mem0 systems rather than labeled approximations of their approach; and a stress test at the scale where Meristem’s bounded-cost design would actually be tested against reality — is the concrete work that stands between the results reported here and the stronger claim they gesture toward. We see this paper as a report on what a specific combination of mechanisms does on the evidence available today, not as a closing argument for it.

References

- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. In *Proceedings of the 28th European Conference on Artificial Intelligence (ECAI 2025)*, 2025.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Sukanta Ganguly. PLACEMEM: Toward a compute-aware memory plane for lifelong agents. *arXiv preprint arXiv:2607.04089*, 2026. Cited to establish that the lifelong-agent-memory space is active and fast-moving as of this writing, not compared against directly.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2024)*, 2024.
- Taher H. Haveliwala. Topic-sensitive PageRank. In *Proceedings of the 11th International Conference on World Wide Web (WWW ’02)*, pages 517–526, 2002. doi: 10.1145/511446.511513.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474, 2020.
- Ripon Chandra Malo and Tong Qiu. PROJECTMEM: A local-first, event-sourced memory and judgment layer for AI coding agents. *arXiv preprint arXiv:2606.12329*, 2026.
- Minns-ai. MinnsDB. <https://github.com/Minns-ai/MinnsDB>, 2026. Accessed 2026-09-12.
- Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. In *Stanford InfoLab Technical Report 1999-66*, 1999. Also presented at the Seventh International World Wide Web Conference (WWW7).
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 2000.
- TencentCloud. TencentDB agent memory. <https://github.com/TencentCloud/TencentDB-Agent-Memory>, 2026. MIT license; v2.0.0 as verified 2026-09-12.